

ITITWE23014

Lê Thành Danh

DSA's Lab - Lab 3 - Stack and Queue

Problem 1.1:

```
import java.util.Stack;

class DeciTOctal {
    public static String decimalToOctal(int decimal) {
        Stack<Integer> stack = new Stack<>();
        StringBuilder octal = new StringBuilder();
        if (decimal == 0) {
            return "0";
        }
        while (decimal > 0) {
            stack.push(decimal % 8);
            decimal = decimal / 8;
        }
        while (!stack.isEmpty()) {
            octal.append(stack.pop());
        }
        return octal.toString();
    }

    public static <T> Stack<T> concatenateStacks(Stack<T> stack1, Stack<T>
stack2) {
        Stack<T> result = new Stack<>();
        while (!stack1.isEmpty()) {
            result.push(stack1.pop());
        }
        while (!stack2.isEmpty()) {
            result.push(stack2.pop());
        }
        return result;
    }

    public static <T> boolean areStacksIdentical(Stack<T> stack1, Stack<T>
stack2) {
        if (stack1.size() != stack2.size()) {
            return false;
        }
    }
}
```

```

    }

    Stack<T> temp1 = (Stack<T>) stack1.clone();
    Stack<T> temp2 = (Stack<T>) stack2.clone();
    while (!temp1.isEmpty()) {
        if (!temp1.pop().equals(temp2.pop())) {
            return false;
        }
    }

    return true;
}

public static void main(String[] args) {
    int decimalNumber = 123;
    System.out.println("Decimal: " + decimalNumber + " => Octal: " +
decimalToOctal(decimalNumber));

    Stack<Integer> stack1 = new Stack<>();
    Stack<Integer> stack2 = new Stack<>();
    stack1.push(1);
    stack1.push(2);
    stack1.push(3);
    stack2.push(4);
    stack2.push(5);
    stack2.push(6);
    Stack<Integer> concatenatedStack = concatenateStacks(stack1,
stack2);
    System.out.println("Concatenated Stack: " + concatenatedStack);
    Stack<Integer> stack3 = new Stack<>();
    Stack<Integer> stack4 = new Stack<>();
    stack3.push(1);
    stack3.push(2);
    stack3.push(3);
    stack4.push(1);
    stack4.push(2);
    stack4.push(3);
    System.out.println("Are stack3 and stack4 identical? " +
areStacksIdentical(stack3, stack4));
    stack4.push(4);
    System.out.println("Are stack3 and stack4 identical after
modification? " + areStacksIdentical(stack3, stack4));
}

```

```
}
```

OutPut:

Concatenated Stack: [3, 2, 1, 6, 5, 4]

Are stack3 and stack4 identical? true

Are stack3 and stack4 identical after modification? False

Explanation:

Decimal to Octal: Use a stack to hold remainders for octal conversion—pop to get the correct order.

Concatenation & Comparison: Merge two stacks, and check if two stacks are alike by comparing their elements.

Problem 1.2:

```
package ITITWE23014_LTDANH_LAB3.Problem_II;
import java.util.Stack;

class SpecialArray {
    private int[] array = new int[20];
    private Stack<int[]> undoStack = new Stack<>();
    private Stack<int[]> redoStack = new Stack<>();

    public SpecialArray() {
        for (int i = 0; i < array.length; i++) {
            array[i] = (int) (Math.random() * 100);
        }
    }

    public void updateValue(int index, int newValue) {
        undoStack.push(array.clone());
        array[index] = newValue;
        redoStack.clear();
    }

    public void undo() {
        if (!undoStack.isEmpty()) {
            redoStack.push(array.clone());
            array = undoStack.pop();
        }
    }
}
```

```

    }

    public void redo() {
        if (!redoStack.isEmpty()) {
            undoStack.push(array.clone());
            array = redoStack.pop();
        }
    }

    public void displayArray() {
        for (int i = 0; i < array.length; i++) {
            System.out.print(array[i] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        SpecialArray specialArray = new SpecialArray();
        specialArray.displayArray();
        specialArray.updateValue(5, 42);
        specialArray.displayArray();
        specialArray.undo();
        specialArray.displayArray();
        specialArray.redo();
        specialArray.displayArray();
    }
}

```

OutPut:

```

47 46 11 30 45 90 14 55 58 14 26 43 17 16 75 35 83 49 71 25
47 46 11 30 45 42 14 55 58 14 26 43 17 16 75 35 83 49 71 25
47 46 11 30 45 90 14 55 58 14 26 43 17 16 75 35 83 49 71 25
47 46 11 30 45 42 14 55 58 14 26 43 17 16 75 35 83 49 71 25

```

Explanation:

Manage changes with undo and redo capabilities, like having a rollback option for your array modifications.

Problem 1.3:

```
// import java.util.Random;
```

```

// class QueueSimulation {

//     static class CircularQueue {
//         private int[] queue;
//         private int front;
//         private int rear;
//         private int maxSize;
//         private int nItems;

//         public CircularQueue(int size) {
//             maxSize = size;
//             queue = new int[size];
//             front = 0;
//             rear = -1;
//             nItems = 0;
//         }

//         public boolean isEmpty() {
//             return nItems == 0;
//         }

//         public boolean isFull() {
//             return nItems == maxSize;
//         }

//         public void insert(int item) {
//             if (isFull()) {
//                 System.out.println("Queue is full. Cannot insert item:
" + item);
//                 return;
//             }
//             rear = (rear + 1) % maxSize;
//             queue[rear] = item;
//             nItems++;
//         }

//         public int remove() {
//             if (isEmpty()) {

```

```

//          System.out.println("Queue is empty. Cannot remove
item.");
//          return -1;
//      }
//      int item = queue[front];
//      front = (front + 1) % maxSize;
//      nItems--;
//      return item;
//  }

//  public void displayQueue() {
//      System.out.println("Queue (Front -> Rear): ");
//      int tempFront = front;
//      for (int i = 0; i < nItems; i++) {
//          System.out.print(queue[tempFront] + " ");
//          tempFront = (tempFront + 1) % maxSize;
//      }
//      System.out.println();
//  }

//  public void displayArrayAndIndices() {
//      System.out.println("\nQueue Array: ");
//      for (int i = 0; i < maxSize; i++) {
//          System.out.print(queue[i] + "\t");
//      }
//      System.out.println("\nFront index: " + front);
//      System.out.println("Rear index: " + rear);
//      System.out.println("Number of items: " + nItems);
//  }

//  public int removeAfterProcessingTime(int n) {
//      if (isEmpty()) {
//          System.out.println("Queue is empty. No item to
remove.");
//          return -1;
//      }

//      try {
//          System.out.println("Processing item: " + queue[front] +
" for " + n + " seconds.");

```

```

//          Thread.sleep(n * 1000);
//          } catch (InterruptedException e) {
//              Thread.currentThread().interrupt();
//          }
//          return remove();
//      }
//  }

//      public static void simulateQueueOperations(CircularQueue queue, int
maxQueueSize, int maxItems, int maxProcessingTime) {
//          Random random = new Random();

//          for (int i = 1; i <= maxItems; i++) {
//              int item = random.nextInt(100);
//              queue.insert(item);
//              System.out.println("Inserted item: " + item);
//          }

//          for (int i = 1; i <= maxItems; i++) {
//              int processingTime = random.nextInt(maxProcessingTime) + 1;
//              int removedItem =
queue.removeAfterProcessingTime(processingTime);
//              System.out.println("Removed item: " + removedItem);

//              queue.displayQueue();
//              queue.displayArrayAndIndices();
//          }

//          System.out.println("Simulation completed.");
//      }

//      public static void main(String[] args) {
//          int queueSize = 5;
//          int maxItems = 10;
//          int maxProcessingTime = 3;

//          CircularQueue queue = new CircularQueue(queueSize);

//          simulateQueueOperations(queue, queueSize, maxItems,
maxProcessingTime);

```

```

//      }
// }

package ITITWE23014_LTDANH_LAB3.Problem_III;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Random;

class QueueSimulation {
    static class Customer {
        String name;
        int serviceTime;

        Customer(String name, int serviceTime) {
            this.name = name;
            this.serviceTime = serviceTime;
        }

        @Override
        public String toString() {
            return "Customer{" + "name='" + name + '\'' + ", serviceTime="
+ serviceTime + '}'';
        }
    }

    static class CircularQueue {
        private Customer[] queue;
        private int front;
        private int rear;
        private int maxSize;
        private int nItems;

        public CircularQueue(int size) {
            maxSize = size;
            queue = new Customer[size];
            front = 0;
            rear = -1;
            nItems = 0;
        }
    }
}

```

```
public boolean isEmpty() {
    return nItems == 0;
}

public boolean isFull() {
    return nItems == maxSize;
}

public void insert(Customer customer) {
    if (isFull()) {
        System.out.println("Queue is full. Cannot insert " +
customer.name);
        return;
    }
    if (rear == maxSize - 1) {
        rear = -1;
    }
    queue[++rear] = customer;
    nItems++;
}

public Customer remove() {
    if (isEmpty()) {
        System.out.println("Queue is empty. No customer to
remove.");
        return null;
    }
    Customer removedCustomer = queue[front++];
    if (front == maxSize) {
        front = 0;
    }
    nItems--;
    return removedCustomer;
}

public void displayQueue() {
    System.out.println("Queue (Front -> Rear): ");
    int tempFront = front;
    for (int i = 0; i < nItems; i++) {
        System.out.println(queue[tempFront]);
    }
}
```

```

        tempFront++;
        if (tempFront == maxSize) {
            tempFront = 0;
        }
    }
}

public void displayArrayAndIndices() {
    System.out.println("\nQueue Array: ");
    for (int i = 0; i < maxSize; i++) {
        System.out.print((queue[i] == null ? "Empty" :
queue[i].name) + "\t");
    }
    System.out.println("\nFront index: " + front);
    System.out.println("Rear index: " + rear);
    System.out.println("Number of items: " + nItems);
}

public Customer removeAfterProcessingTime() {
    if (isEmpty()) {
        System.out.println("Queue is empty. No customer to
remove.");
        return null;
    }
    Customer customer = queue[front];
    try {
        System.out.println("Serving " + customer.name + " for " +
customer.serviceTime + " seconds.");
        Thread.sleep(customer.serviceTime * 1000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    return remove();
}

public static void simulateQueueProcessing(CircularQueue queue, int
maxQueueSize, int maxCustomers,
    int maxServiceTime, int customerArrivalRate) {
    Random random = new Random();

```

```

        int customersArrived = 0;
        int timeElapsed = 0;
        while (customersArrived < maxCustomers) {
            if (!queue.isFull()) {
                String customerName = "Customer_" + (customersArrived +
1);

                int serviceTime = random.nextInt(maxServiceTime) + 1;
                Customer newCustomer = new Customer(customerName,
serviceTime);

                queue.insert(newCustomer);
                System.out.println("Customer arrived: " + newCustomer);
                customersArrived++;
            }
            if (!queue.isEmpty()) {
                queue.removeAfterProcessingTime();
            }
            try {
                int waitTime = random.nextInt(customerArrivalRate) + 1;
                Thread.sleep(waitTime * 1000);
                timeElapsed += waitTime;
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            queue.displayQueue();
            queue.displayArrayAndIndices();
        }
        System.out.println("Simulation done");
    }

    public static void main(String[] args) {
        int queueSize = 5;
        int maxCustomers = 10;
        int maxServiceTime = 5;
        int customerArrivalRate = 3;
        CircularQueue queue = new CircularQueue(queueSize);
        simulateQueueProcessing(queue, queueSize, maxCustomers,
maxServiceTime, customerArrivalRate);
    }
}

```

OutPut:

Serving Customer_1 for 3 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_1 Empty Empty Empty Empty

Front index: 1

Rear index: 0

Number of items: 0

Customer arrived: Customer{name='Customer_2', serviceTime=2}

Serving Customer_2 for 2 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_1 Customer_2 Empty Empty Empty

Front index: 2

Rear index: 1

Number of items: 0

Customer arrived: Customer{name='Customer_3', serviceTime=2}

Serving Customer_3 for 2 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_1 Customer_2 Customer_3 Empty Empty

Front index: 3

Rear index: 2

Number of items: 0

Customer arrived: Customer{name='Customer_4', serviceTime=4}

Serving Customer_4 for 4 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_1 Customer_2 Customer_3 Customer_4 Empty

Front index: 4

Rear index: 3

Number of items: 0

Customer arrived: Customer{name='Customer_5', serviceTime=4}

Serving Customer_5 for 4 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_1 Customer_2 Customer_3 Customer_4 Customer_5

Front index: 0

Rear index: 4

Number of items: 0

Customer arrived: Customer{name='Customer_6', serviceTime=4}

Serving Customer_6 for 4 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_6 Customer_2 Customer_3 Customer_4 Customer_5

Front index: 1

Rear index: 0

Number of items: 0

Customer arrived: Customer{name='Customer_7', serviceTime=5}

Serving Customer_7 for 5 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_6 Customer_7 Customer_3 Customer_4 Customer_5

Front index: 2

Rear index: 1

Number of items: 0

Customer arrived: Customer{name='Customer_8', serviceTime=5}

Serving Customer_8 for 5 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_6 Customer_7 Customer_8 Customer_4 Customer_5

Front index: 3

Rear index: 2

Number of items: 0

Customer arrived: Customer{name='Customer_9', serviceTime=1}

Serving Customer_9 for 1 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_6 Customer_7 Customer_8 Customer_9 Customer_5

Front index: 4

Rear index: 3

Number of items: 0

Customer arrived: Customer{name='Customer_10', serviceTime=5}

Serving Customer_10 for 5 seconds.

Queue (Front -> Rear):

Queue Array:

Customer_6 Customer_7 Customer_8 Customer_9

Customer_10

Front index: 0

Rear index: 4

Number of items: 0

Simulation done

Explanation:

A queue that wraps around to efficiently add and remove items, akin to a continual amusement park line, the horsy one.

Problem 1.4:

```
package ITITWE23014_LTDANH_LAB3.Problem_IV;

import java.util.Stack;

class Person {
    String name;
    Integer age;

    Person(String name, Integer age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return name + ", " + age + " yrs old";
    }
}

class ReverseApp {
    public static void reversePersonsList(Stack<Person> stack) {
        if (stack.isEmpty()) {
            return;
        }
        Person person = stack.pop();
        reversePersonsList(stack);
        insertAtBottom(stack, person);
    }

    private static void insertAtBottom(Stack<Person> stack, Person person)
    {
        if (stack.isEmpty()) {
            stack.push(person);
        } else {
            Person top = stack.pop();

```

```

        insertAtBottom(stack, person);
        stack.push(top);
    }
}

public static void main(String[] args) {
    Stack<Person> persons = new Stack<>();
    persons.push(new Person("John", 20));
    persons.push(new Person("Alice", 30));
    persons.push(new Person("Bob", 42));
    System.out.println("Original Stack:");
    for (Person person : persons) {
        System.out.println(person);
    }
    reversePersonsList(persons);
    System.out.println("\nReversed Stack:");
    for (Person person : persons) {
        System.out.println(person);
    }
}
}

```

OutPut:

```

John, 20 yrs old
Alice, 30 yrs old
Bob, 42 yrs old

```

```

Reversed Stack:
Bob, 42 yrs old
Alice, 30 yrs old
John, 20 yrs old

```

Explanation:

Flip a stack by using a recursive method to reverse the order of elements by popping elements until the stack is empty.

As the recursion unwinds, insert each element at the bottom of the stack, effectively reversing the order.

Problem 1.5:

```

package ITITWE23014_LTDANH_LAB3.Problem_V;

```

```

import java.util.PriorityQueue;

class Queue {
    private int maxSize;
    private long[] queArray;
    private int front;
    private int rear;
    private int nItems;

    //-----
    public Queue(int s) {          // constructor
        maxSize = s;
        queArray = new long[maxSize];
        front = 0;
        rear = -1;
        nItems = 0;
    }

    //-----
    public void insert(long j) {    // put item at rear of queue
        if (rear == maxSize - 1) { // deal with wraparound
            rear = -1;
        }
        queArray[++rear] = j;       // increment rear and insert
        nItems++;                   // one more item
    }

    //-----
    public long remove() {          // take item from front of queue
        long temp = queArray[front++]; // get value and incr front
        if (front == maxSize) {      // deal with wraparound
            front = 0;
        }
        nItems--;                   // one less item
        return temp;
    }

    //-----
    public long peekFront() {       // peek at front of queue

```

```

        return queArray[front];
    }

    //-----
    public boolean isEmpty() {    // true if queue is empty
        return (nItems == 0);
    }

    //-----
    public boolean isFull() {    // true if queue is full
        return (nItems == maxSize);
    }

    //-----
    public int size() {          // number of items in queue
        return nItems;
    }

    //-----
    // Display the current state of the queue
    public void displayQueue() {
        System.out.print("Queue: ");
        if (isEmpty()) {
            System.out.println("Empty");
            return;
        }
        for (int i = 0; i < nItems; i++) {
            int index = (front + i) % maxSize;
            System.out.print(queArray[index] + " ");
        }
        System.out.println();
    }
}

class QueueApp {
    public static void main(String[] args) {
        Queue theQueue = new Queue(5);    // queue holds 5 items

        // Insert items into the queue

```

```

        theQueue.insert(10);
        theQueue.insert(20);
        theQueue.insert(30);
        theQueue.insert(40);

        // Display the queue state after insertions
        System.out.println("Queue after insertions:");
        theQueue.displayQueue();

        // Remove 3 items
        System.out.println("Removing items:");
        theQueue.remove(); // 10
        theQueue.remove(); // 20
        theQueue.remove(); // 30

        // Display the queue state after removals
        theQueue.displayQueue();

        // Insert more items (with wraparound)
        theQueue.insert(50);
        theQueue.insert(60);
        theQueue.insert(70);
        theQueue.insert(80);

        // Display the queue state after additional insertions
        System.out.println("Queue after more insertions:");
        theQueue.displayQueue();

        // Remove and display all items
        System.out.println("Removing all items:");
        while (!theQueue.isEmpty()) {
            long n = theQueue.remove();
            System.out.print(n + " ");
        }
        System.out.println();
    }
}

class PriorityQueueApp {
    public static void main(String[] args) {

```

```

        PriorityQueue<Long> pq = new PriorityQueue<>();

        pq.add(40L);
        pq.add(10L);
        pq.add(30L);
        pq.add(20L);

        System.out.println("Priority Queue state:");
        while (!pq.isEmpty()) {
            System.out.print(pq.poll() + " ");
            System.out.println();
        }
    }
}

```

OutPut:

QueueApp

Queue after insertions:

Queue: 10 20 30 40

Removing items:

Queue: 40

Queue after more insertions:

Queue: 40 50 60 70 80

Removing all items:

40 50 60 70 80

PriorityQueueApp

Priority Queue state:

10 20 30 40

Explanation:

Queue: Mimics a straightforward line at a cafe.

When I insert new elements, I add them to the rear, managing wraparounds to maintain the circular effect.

For removals, I take elements from the front and shift the front index forward, again using wraparound for seamless operation.

PriorityQueue: Sorts based on importance, allowing urgent elements to leap ahead.

I use Java's PriorityQueue to automatically organize elements based on their natural ordering or a specified comparator.

built-in PriorityQueue is implementing a min-heap by default, so it's more efficient i think.

Problem 1.6

```
package ITITWE23014_LTDANH_LAB3.Problem_VI;

import java.util.LinkedList;
import java.util.Queue;

class CustomerServiceCenter {
    private Queue<String> regularQueue;
    private Queue<String> vipQueue;

    public CustomerServiceCenter() {
        regularQueue = new LinkedList<>();
        vipQueue = new LinkedList<>();
    }

    public void enqueue(String customerName, boolean isVip) {
        if (isVip) {
            vipQueue.offer(customerName);
        } else {
            regularQueue.offer(customerName);
        }
    }

    public void serveCustomer() {
        if (!vipQueue.isEmpty()) {
            System.out.println("Serve " + vipQueue.poll());
        } else if (!regularQueue.isEmpty()) {
            System.out.println("Serve " + regularQueue.poll());
        } else {
            System.out.println("No customers in the queue");
        }
    }

    public static void main(String[] args) {
        CustomerServiceCenter serviceCenter = new CustomerServiceCenter();
        serviceCenter.enqueue("Alice", false);
        serviceCenter.enqueue("Bob", true);
        serviceCenter.enqueue("Charlie", false);
        serviceCenter.serveCustomer();
    }
}
```

```

        serviceCenter.serveCustomer() ;
        serviceCenter.serveCustomer() ;
        serviceCenter.serveCustomer() ;
    }
}

```

OutPut:

```

Serve Bob
Serve Alice
Serve Charlie
No customers in the queue

```

Explanation:

Prioritize VIP customers with two separate queues to ensure high-value clients are served first.

Data Structure:

Two queues are used:

vipQueue: Holds VIP customers.

regularQueue: Holds regular customers.

Both queues are implemented using **LinkedList**, which supports efficient enqueue and dequeue operations.

Algorithm:

enqueue(String customerName, boolean isVip):

Adds the customer to the VIP queue if isVip is true, or to the regular queue if false.

serveCustomer():

First serves a customer from the VIP queue if available.

If no VIP customers, serve from the regular queue.

If both queues are empty, print "No customers in the queue".

Time Complexity:

enqueue: $O(1)$ – adding a customer to the queue.

serveCustomer: $O(1)$ – removing the customer from the front of the queue.